

PROGRAMACIÓN APLICADA 2026-2

CAPÍTULO 01

JAVA 21 ESENCIAL PARA COMENZAR EL BACKEND

GUÍA DEL ESTUDIANTE

Aprende con ParkFlow 360 y transfiere el patrón a tu propio proyecto

Proyecto espejo del docente: ParkFlow 360

Entidades de referencia: CLIENTE → VEHÍCULO (1:N)

Propósito del capítulo: dominar únicamente el Java que necesitaremos para comprender Spring Boot y la arquitectura hexagonal. Todavía NO usamos Spring Boot, JPA, controladores ni repositorios.

1. Qué vas a aprender

Tu docente utilizará Cliente y Vehículo de ParkFlow 360 como ejemplo. Tú NO debes copiar esas entidades. Debes identificar en tu proyecto asignado dos tablas propias con una relación 1:N semejante y construir su primera representación en Java 21.

Regla central: Observa el patrón del profesor → comprende por qué funciona → reproduce la misma ingeniería con las tablas de TU proyecto.

2. Qué NO haremos todavía

- No crearemos Spring Boot.
- No usaremos @Entity.
- No usaremos JpaRepository.
- No conectaremos Java con PostgreSQL.
- No implementaremos las 16 tablas.
- No haremos CRUD completo.

Primero aprenderás a expresar conceptos del negocio mediante clases Java. Esta base hará que Spring Boot sea comprensible en lugar de una colección de anotaciones.

3. Cómo crear y organizar TU proyecto en IntelliJ IDEA

Antes de programar tus dos entidades, debes construir una estructura de proyecto limpia. El profesor utilizará ParkFlow 360 como ejemplo, pero tú debes derivar los nombres del proyecto, package base y módulos a partir del proyecto oficial asignado a tu grupo. No copies com.parkflow ni los paquetes customer/vehicle si esos conceptos no existen en tu dominio.

3.1. Regla de transferencia: profesor → tu proyecto

El profesor mostrará: parkflow-backend-lab → com.parkflow → customer.domain / vehicle.domain. Tú debes conservar el PATRÓN de organización, pero reemplazar cada nombre por uno coherente con tu proyecto y con las dos entidades 1:N seleccionadas.

Ejemplo de traducción:

```
Profesor: parkflow-backend-lab
Package base: com.parkflow
Entidad padre: Cliente → package customer.domain
Entidad dependiente: Vehículo → package vehicle.domain

Tu grupo: <nombre-proyecto>-backend-lab
Package base: com.<nombreproyecto>
Entidad padre: <tu entidad A> → package <móduloA>.domain
Entidad dependiente: <tu entidad B> → package <móduloB>.domain
```

3.2. Cómo debes llamar a tu proyecto

Usa esta convención obligatoria para el laboratorio de Java de los capítulos 01 y 02:

```
<nombre-proyecto>-backend-lab
```

Reglas: todo en minúsculas; palabras separadas con guion; sin espacios; sin tildes; sin ñ; debe identificar claramente el proyecto. Ejemplos: farmastock-backend-lab, bibliosmart-backend-lab, cargotrack-backend-lab. Cuando creamos el proyecto Spring Boot real, utilizaremos <nombre-proyecto>-backend.

3.3. Cómo debes llamar al package base

Para mantener una convención homogénea en la materia, usa:

```
com.<nombreproyecto>
```

El identificador <nombreproyecto> debe ir en minúsculas y sin guiones, espacios, tildes ni caracteres especiales. Por ejemplo, un proyecto llamado FarmaStock 360 puede utilizar com.farmastock; BiblioSmart puede utilizar com.bibliosmart. Esta es una convención académica del curso para aprender estructura de packages.

NO uses nombres como com.parkflow si tu proyecto no es ParkFlow. Tampoco uses package1, proyecto, prueba, clases o similares.

3.4. Crear el proyecto Java 21 en IntelliJ IDEA - paso a paso

1. Abrir IntelliJ IDEA.
2. Seleccionar New Project.
3. Elegir Java como tipo de proyecto.
4. En Name escribir el nombre definido por tu grupo: <nombre-proyecto>-backend-lab.
5. Elegir una carpeta de trabajo conocida y permanente.
6. En JDK seleccionar Java 21.
7. NO seleccionar Spring Boot todavía.
8. NO agregar JPA, Lombok, PostgreSQL Driver ni otros frameworks.
9. Crear el proyecto.
10. Identificar el source root: src si es un proyecto Java simple, o src/main/java si IntelliJ/Maven lo ha creado así.

Comprueba el JDK desde la Terminal de IntelliJ:

```
java -version
javac -version
```

3.5. ¿Dónde crear los packages?

Todos los packages Java deben crearse DENTRO del source root. Si trabajas con src/main/java, haz clic derecho sobre java. Si trabajas con un proyecto Java simple, haz clic derecho sobre src. Nunca crees los packages dentro de .idea, resources, target o fuera del source root.

3.6. Identifica tus dos módulos antes de crear carpetas

Recupera las dos tablas 1:N seleccionadas en tu DER físico. Define cuál será tu entidad padre A y cuál la entidad dependiente B. Después convierte los nombres a un nombre de módulo Java en minúsculas y, preferentemente, en inglés técnico o en una convención consistente para todo el grupo.

Completa antes de continuar:

- Proyecto oficial: _____

- Nombre del proyecto IntelliJ: _____
- Package base: _____
- Tabla/entidad padre A: _____
- Nombre de módulo A: _____
- Tabla/entidad dependiente B: _____
- Nombre de módulo B: _____
- FK que conecta B con A: _____

3.7. Estructura exacta que debes construir

```

<nombre-proyecto>-backend-lab
├── src/main/java (o src si es Java simple)
│   └── com
│       └── <nombreproyecto>
│           ├── Main.java
│           ├── <móduloA>
│           │   ├── domain
│           │   │   ├── TuEntidadPadre.java
│           │   │   ├── TuEntidadRepository.java
│           │   │   └── TuEntidadResumen.java (cuando corresponda)
│           │   └── exception
│           │       └── ...
│           └── <móduloB>
│               ├── domain
│               │   ├── TuEntidadDependiente.java
│               │   └── TuEnum.java (si tu dominio lo necesita)
│               └── exception
│                   └── ...

```

No tienes que crear todos los archivos vacíos el primer minuto. Debes crear primero la estructura de packages y luego ir agregando las clases exactamente cuando la guía las introduzca.

3.8. Ejemplo de cómo transferir el patrón

Supón que tu proyecto tiene la relación PEDIDO 1:N DETALLE_PEDIDO. Una posible estructura sería:

```

com.miproyecto
├── Main.java
├── order
│   ├── domain
│   │   ├── Pedido.java
│   │   └── PedidoRepository.java
│   └── exception
└── orderdetail
    ├── domain
    │   └── DetallePedido.java
    └── exception

```

El ejemplo anterior enseña el patrón, no obliga esos nombres. Debes justificar tus nombres con el lenguaje de tu dominio.

3.9. Cómo crear cada package en IntelliJ IDEA

1. En el panel Project, localiza src/main/java (o src).
2. Clic derecho → New → Package.
3. Escribe tu package base completo, por ejemplo: com.farmastock.
4. Clic derecho sobre el package base → New → Package → escribe <móduloA>.domain.

5. Repite → <moduloA>.exception.
6. Crea <moduloB>.domain.
7. Crea <moduloB>.exception.
8. Clic derecho sobre el package base → New → Java Class → crea Main.
9. En <moduloA>.domain crea la clase de tu entidad padre cuando llegue el paso correspondiente.
10. En <moduloB>.domain crea la clase de tu entidad dependiente.
11. Crea enums, interfaces, records y excepciones solamente cuando la guía los solicite.

3.10. Convenciones obligatorias de nombres

- Proyecto/carpeta: kebab-case → farmastock-backend-lab.
- Packages: minúsculas, sin guiones → com.farmastock.product.domain.
- Clases, interfaces, records y enums: PascalCase → Producto, ProductoRepository, ProductoResumen, EstadoProducto.
- Métodos y variables: camelCase → registrarProducto(), productoId, fechaRegistro.
- Constantes de enum: MAYÚSCULAS_CON_GUIÓN_BAJO → PENDIENTE_REVISION.
- Un archivo Java debe tener el mismo nombre que su clase pública.
- Evita paquetes genéricos globales como model, entity, service o utils para todo el sistema; organizaremos el backend por módulos de negocio.

3.11. Qué significa package y por qué importa

Cuando IntelliJ crea una clase dentro de tu módulo, debe aparecer una declaración semejante a:

```
package com.<nombreproyecto>.<moduloA>.domain;

public class TuEntidadPadre {
}
```

El package indica la ubicación lógica de la clase dentro del código. No es una tabla, no es un schema de PostgreSQL y no crea una relación de base de datos. Sirve para organizar tipos Java y evitar colisiones de nombres.

3.12. Imports entre módulos

Si tu entidad padre necesita utilizar el tipo de la entidad dependiente, las clases estarán en packages diferentes y Java necesitará un import. IntelliJ normalmente lo agrega automáticamente:

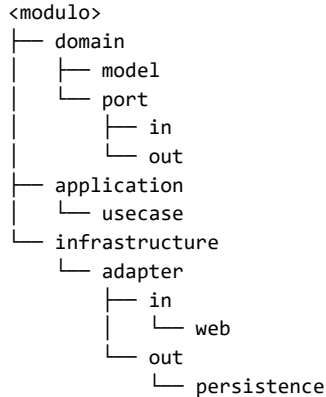
```
import com.<nombreproyecto>.<moduloB>.domain.TuEntidadDependiente;
import java.util.ArrayList;
import java.util.List;
```

Un import NO copia la clase, NO crea una FK y NO conecta PostgreSQL. Sólo permite utilizar desde una clase un tipo Java declarado en otro package.

3.13. Qué NO debes crear todavía

No adelantes la arquitectura. En los capítulos 01 y 02 NO crees todavía controller, application, usecase, adapter, infrastructure, JpaRepository, @Entity ni configuraciones de Spring. Esos packages aparecerán cuando puedas explicar para qué sirven.

La evolución futura de un módulo será aproximadamente:



3.14. Punto de control obligatorio antes de programar

- ☐ Mi proyecto NO se llama parkflow-backend-lab, salvo que realmente mi proyecto sea ParkFlow.
- ☐ El nombre de mi proyecto sigue <nombre-proyecto>-backend-lab.
- ☐ IntelliJ utiliza JDK 21.
- ☐ Identifiqué correctamente el source root.
- ☐ Mi package base representa mi propio proyecto.
- ☐ Creé un módulo/package para mi entidad padre.
- ☐ Creé un módulo/package para mi entidad dependiente.
- ☐ Puedo explicar qué significa domain y exception.
- ☐ Puedo explicar la diferencia entre carpeta física y package Java.
- ☐ No creé todavía Spring Boot, JPA, controller ni repositorios de persistencia.

4. El ejemplo que verás: Cliente → Vehículo

Modelo físico ParkFlow	Interpretación para Java
CLIENTE	Concepto principal/padre
VEHICULO	Concepto dependiente
FK vehiculo.cliente_id	Un cliente puede tener varios vehículos
UNIQUE placa	Regla de unicidad persistente

CHECK tipo	Conjunto controlado; en Java puede inspirar un enum
------------	---

Importante: La FK existe en PostgreSQL. En Java la relación se puede expresar mediante objetos y colecciones. No son representaciones idénticas.

5. Paso 1 - elige tus dos tablas

Busca en tu DER físico una relación 1:N simple. Completa antes de programar:

Elemento	Tu respuesta
Tabla A (padre)	_____
PK de A	_____
Tabla B (dependiente)	_____
PK de B	_____
FK de B hacia A	_____
Regla sencilla que quieres proteger	_____

Ejemplos de patrones válidos (no son para copiar): Pedido→Detalle, Cliente→Dirección, Categoría→Producto, Curso→Sección, Proyecto→Tarea.

6. Paso 2 - transforma la tabla A en una clase Java

Tu profesor mostrará algo como:

```
public class Cliente {
    private Long id;
    private String nombreCompleto;
    private String documento;
    private String email;

    public Cliente(Long id, String nombreCompleto, String documento, String email) {
        this.id = id;
        this.nombreCompleto = nombreCompleto;
        this.documento = documento;
        this.email = email;
    }
}
```

Tu trabajo es construir la clase correspondiente a TU tabla A. No copies nombres de ParkFlow.

```
public class TuEntidadPadre {
    // 1. atributos principales
    // 2. constructor
    // 3. métodos de lectura necesarios
    // 4. comportamiento propio
}
```

7. Paso 3 - encapsula: evita public y setters indiscriminados

Los atributos deben ser private para evitar cambios arbitrarios desde cualquier parte del programa. El objeto debe controlar cómo cambia su propio estado.

Evita	Prefiere
public String estado;	private Estado estado;
setEstado("cualquier cosa")	método de negocio con regla
lista pública	colección privada + método agregar...()

8. Paso 4 - crea la clase B

Tu profesor modelará Vehiculo. Tú debes modelar la tabla B de tu proyecto usando los atributos que realmente existen en tu diseño físico.

```
public class TuEntidadDependiente {
    private Long id;
    // agrega aquí 3-6 atributos de tu tabla B

    public TuEntidadDependiente(/* parámetros */) {
        // valida al menos una regla simple
    }
}
```

9. Paso 5 - usa enum cuando el conjunto de valores sea cerrado

Si en PostgreSQL tienes un CHECK con estados controlados, evalúa representarlo en Java con enum.

```
public enum EstadoEjemplo {
    PENDIENTE,
    ACTIVO,
    CERRADO
}
```

No inventes: Tu enum debe salir de una regla real de TU proyecto, no del ejemplo de ParkFlow.

10. Paso 6 - representa la relación 1:N

El profesor usará una List<Vehiculo> dentro de Cliente. Tú debes implementar una colección del tipo B dentro de tu entidad A, si esa navegación tiene sentido en tu dominio.

```
private final List<TuEntidadDependiente> elementos = new ArrayList<>();

public void agregarElemento(TuEntidadDependiente elemento) {
    // 1. valida una regla
    // 2. agrega el elemento
    elementos.add(elemento);
}

public List<TuEntidadDependiente> getElementos() {
    return Collections.unmodifiableList(elementos);
}
```


No debes copiar el nombre `agregarVehiculo()`. Tu método debe usar lenguaje real de tu negocio: `agregarDetalle()`, `asignarTarea()`, `registrarDireccion()`, `agregarProducto()`, etc.

11. Paso 7 - crea un Main de prueba

```
public class Main {
    public static void main(String[] args) {
        // 1. crea un objeto padre
        // 2. crea dos objetos dependientes
        // 3. relacionalos mediante el método de negocio
        // 4. imprime una evidencia
    }
}
```

Tu programa debe ejecutarse sin Spring. Si no puedes explicar este flujo, todavía no estás listo para avanzar a Spring Boot.

12. Paso 8 - primera interface: pensar en contratos

El profesor mostrará `ClienteRepository` como contrato. Tú debes crear uno equivalente para tu entidad principal.

```
public interface TuEntidadRepository {
    TuEntidadPadre guardar(TuEntidadPadre entidad);
    Optional<TuEntidadPadre> buscarPorId(Long id);
}
```

Atención: No implementes todavía PostgreSQL ni `JpaRepository`. En este capítulo la interface sólo expresa QUÉ necesitamos, no CÓMO se hará.

13. Paso 9 - compara PostgreSQL con Java

Pregunta	Debes poder responder con tu proyecto
¿Cuál es la tabla padre?	_____
¿Cuál es la tabla dependiente?	_____
¿Dónde está la FK en PostgreSQL?	_____
¿Cómo representaste el 1:N en Java?	_____
¿Qué regla protege tu método?	_____
¿Qué enum creaste y de qué CHECK/regla proviene?	_____

14. Verificación de la estructura de packages

```
<nombre-proyecto>-backend-lab
├── src/main/java (o src)
│   └── com/<nombreproyecto>/
│       ├── Main.java
│       ├── <moduloA>/
│       │   ├── domain/
│       │   └── exception/
│       └──
```

```

└─ <moduloB>/
   └─ domain/
      └─ exception/

```

Esta estructura todavía no es la arquitectura hexagonal final. Es un laboratorio para aprender Java sin ruido adicional.

15. Entrega técnica en Git

- Tus dos clases Java.
- Constructor en ambas clases.
- Atributos private.
- Una relación 1:N mediante colección si corresponde.
- Un enum real del dominio cuando corresponda.
- Una regla validada desde Java.
- Una interface de contrato.
- Main ejecutable.
- README con la tabla A, tabla B, PK, FK y cómo lo representaste en Java.

```

git add .
git commit -m "feat: crear modelo Java inicial del dominio"

```

16. Plantilla del README

```

# Capítulo 01 - Java esencial

## Entidades elegidas
- Tabla padre: ...
- Tabla dependiente: ...
- Relación: 1:N
- FK: ...

## Clases Java
- ...
- ...

## Regla implementada
...

## Decisiones
- ¿Por qué usamos enum? ...
- ¿Por qué la colección es privada? ...
- ¿Qué NO implementamos todavía? Spring Boot, JPA y PostgreSQL desde Java.

```

17. Autocomprobación antes de entregar

- ☐ Puedo explicar cada línea de mis dos clases.
- ☐ Sé distinguir tabla, fila, clase y objeto.
- ☐ Sé explicar por qué los atributos son private.
- ☐ Sé crear dos objetos y relacionarlos.
- ☐ Sé explicar qué significa 1:N en PostgreSQL y en Java.
- ☐ No utilicé @Entity, Spring, JPA ni repositorios reales.

- ☐ Mi código compila y ejecuta.
- ☐ Mi commit está en Git.

18. Preguntas para tu defensa individual

- Muéstrame cuál de tus clases representa la entidad padre.
- Muéstrame dónde representaste la relación 1:N.
- ¿Por qué no hiciste los atributos public?
- ¿Qué pasaría si devolvieras la lista mutable directamente?
- ¿Qué regla valida tu constructor o método?
- ¿Por qué tu enum representa mejor el dominio que un String libre?
- ¿Qué declara tu interface?
- ¿Tu interface ya sabe usar PostgreSQL? ¿Por qué no?
- ¿Qué parte de tu diseño físico todavía NO aparece en este capítulo?

19. Definition of Done

Puedes avanzar cuando: tu código Java representa correctamente dos conceptos reales de tu proyecto, la relación entre ellos, al menos una regla, y puedes defender todas las decisiones sin depender de copiar el ejemplo de ParkFlow.

20. Qué viene después

En el Capítulo 02 profundizaremos las herramientas de Java que Spring utiliza constantemente: interfaces, colecciones, Optional, excepciones propias, enums, records y packages. Mantendrás esta misma estructura basada en TU proyecto. Después construiremos el primer proyecto Spring Boot real.